Automated Bootstrapping Placement for Encrypted Inference

CS5491 Artificial Intelligence Project Milestone

Member 1: Xing Chunjie | Student ID: 72655910
Member 2: Yang Mianji | Student ID: 72655251
Member 3: Zhang Qi | Student ID: 72655380
Contact email: 72655910@cityu-dg.edu.cn

1. Problem and motivation

We study where to insert bootstrapping in the supplied multilayer perceptron (MLP) and attention computation graphs, including their LayerNorm and Softmax operations. In the course model of encrypted inference, ciphertext levels represent the remaining capacity for multiplication. Bootstrapping restores levels but adds substantial latency; Modswitch lowers a level at zero modelled cost. Placement decisions interact across branches: one refresh can serve several consumers, and levels also affect ordinary operator costs. Minimising refresh count alone can therefore miss a faster placement. We minimise total estimated inference latency while preserving the computation and satisfying every operator's level requirements.

The scope is two fixed graphs, each at input levels 0 through 14 with maximum level 14: 30 conditions, not 30 different networks. Original operators, edges and ciphertext counts are preserved, and there is no required output level. Bootstrap1 (B1) restores level 14 at 1040.15 ms per ciphertext. Bootstrap2 (B2) restores level 13 at 1293.13 ms per pair and is allowed only for tensors of 8 or 16 ciphertexts.

2. Joint placement model

We use OR-Tools CP-SAT, a solver for integer and logical constraints, to choose levels and insertions jointly. Finite integer levels, discrete refresh choices and shared costs fit this representation. The instructor provides the graphs, latency functions and scorer; OR-Tools provides the general search engine. Our contribution is the shared-refresh formulation, graph reconstruction and validation workflow. We do not claim that CP-SAT outperforms untested search methods.

For each operator v, the model chooses input and output levels a_v and b_v. It enforces the valid input range and b_v = a_v - d_v, where d_v is the operator's level consumption. On each edge, it chooses no refresh, B1 or B2, and a delivered level no greater than the available level. A lower delivery implies a strict Modswitch. Block inputs are fixed to the selected input level; at a merge, the receiving operator uses the minimum delivered predecessor level.

One activation variable per producer and refresh type records whether any outgoing edge uses that refresh. The objective charges each activated refresh once, together with every ordinary operator's level-dependent cost. Both refresh types may be used after the same producer; B2 is disabled on ineligible tensors. The validated teacher placement supplies search hints and a feasible cost upper bound, which cannot exclude a better solution.

3. Reconstruction and correctness checks

A solver assignment becomes an actual directed acyclic graph (DAG) with inserted operations. The permitted edge chains are empty, B1, B2, M, B1-M and B2-M, where M denotes Modswitch. The chosen refresh mode and delivered level determine one of these chains; conversely, each permitted chain determines those choices. Reconstruction shares refreshes by producer and type, and shares identical switches by producer, refresh mode and target level. It inserts a switch only for a strict level drop. Thus shared operations are represented and charged once rather than duplicated for each consumer.

Integer objective coefficients are scaled by 10000 to preserve the supplied four-decimal costs. A conservative bound on expression magnitudes is checked below 2^53, and both solver gap tolerances are zero. For each saved solution, we checked graph preservation, ciphertext counts, allowed chains, shared prefixes, integer levels, operator domains, merges and B2 eligibility. Independent exact-decimal costs were compared with the unchanged instructor scorer at a tolerance of 1e-7 ms, alongside levels, refresh counts and the other required metrics. A score alone is not treated as proof of legality.

The completed validation suite passed 44 tests, including failure paths and falsified summaries. Six tiny graphs were exhaustively enumerated; their results agreed with hand arithmetic, CP-SAT, reconstruction, audit and scoring. Enumeration does not invoke CP-SAT or reconstruction, but modelling, audit and enumeration share a rules table; enumeration and model building also share original-graph checks. They are not independent interpretations of the requirements. These are correctness checks on small examples, not a second full-scale optimisation benchmark.

4. Preliminary comparison protocol

The saved comparison was executed on 28 September 2026 using CPython 3.13.9, OR-Tools 9.15.6755, one CPU search worker, seed 5491 and a 60-second limit per condition. All 19 dependencies are pinned. The evidence package identifies the exact execution revision and source fingerprint, configurations and saved placements; a later report revision does not replace the executed version.

All 30 instructor baselines were calibrated. Each optimised graph is compared with the same block and input level, using all ordinary operator costs and all distinct inserted-operation costs. The instructor baseline is a fixed placement that performs no placement search, so its scoring time is not a competing optimiser's search time. We report estimated inference latency separately from measured solve time and memory. The results and checks below are preserved preliminary evidence; the final experimental protocol and independent reproduction remain outstanding.

5. Preliminary results

All 30 saved placements passed structural and level checks, strictly improved their same-input baselines and returned OPTIMAL with matching objective values and lower bounds. These bounds establish optimality within the encoded course model, not measured encrypted-inference performance. No course condition failed, timed out or used a baseline fallback. All 18 conditions shared with the earlier prototype have identical scores; the remaining 12 attention conditions complete coverage.

Block, inputs	Latency (ms)	Reduction (%)	B1 / B2
MLP, 0-13	26069.9524	7.3223-7.3291	4 / 4
MLP, 14	21911.5924	22.1113	0 / 4
Attention, 0-5	45698.2664	14.6755	10 / 8
Attention, 6-14	41537.6664	22.4438	6 / 8

Table 1. Equal-cost ranges summarise all 30 conditions. B1 counts ciphertexts; B2 counts pairs. Shared operations are charged once. Full per-input records retain each placement and all required metrics.

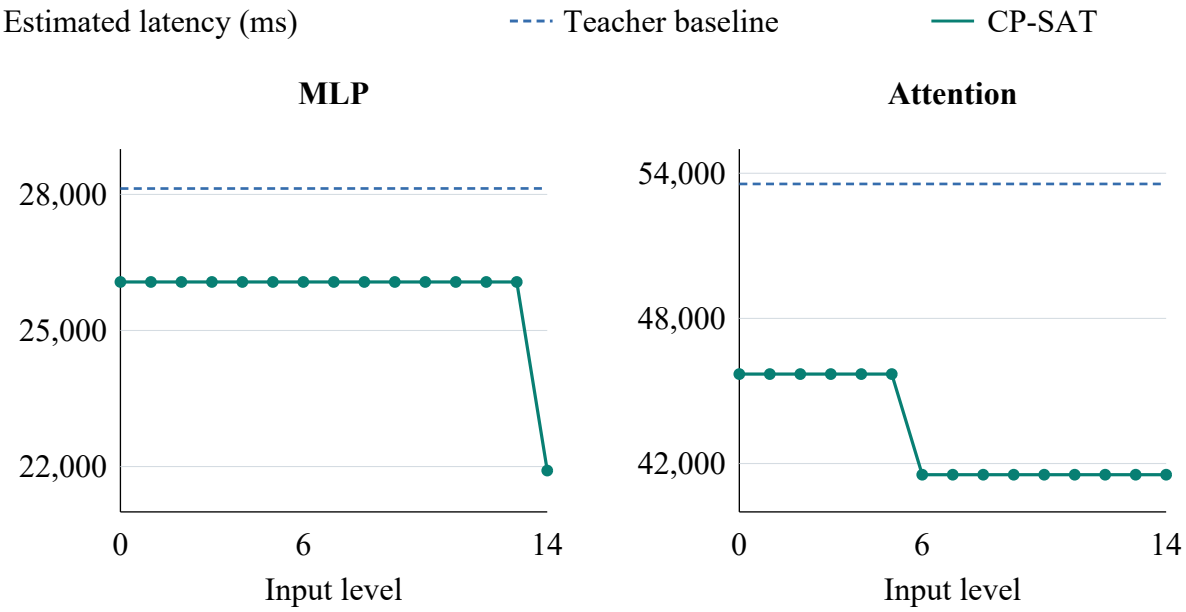


Figure 1. Estimated inference latency for both blocks across all input levels. Lines connect integer inputs for readability. These fitted costs are neither actual encrypted-inference measurements nor solver times.

Depth, the maximum original level consumption along a path, is 26 for MLP and 62 for attention. The course budget is 14*B1 + 13*B2 + input level; utilisation is depth/budget. Neither metric replaces the full-cost objective.

Measured solve calls total 10.6445 seconds: 0.0075-0.0132 per MLP condition and 0.5166-0.9656 per attention condition. Maximum sampled process resident set size (RSS) is 118.36 MiB. The 20 ms samples include imported libraries and may miss a peak. These are one local Windows AMD64 run's observations, not end-to-end times or cross-device guarantees.

6. What the saved placements explain

Attention's lower-cost range begins at input level 6. Comparing its saved input-0 and input-6 solutions, the only difference in nonzero operation costs is the removal of an early B1 on four ciphertexts, worth 4160.60 ms. At input 6, the query and key projections each consume two levels and deliver level 4 to the next ciphertext multiplication. The available input level therefore avoids that early refresh in the saved optimum. Individual solver bounds establish the optimum at each input; comparing placements explains the cost change.

MLP shows a similar change at input 14. Relative to input 0, its saved solution removes an early B1, saving 4160.60 ms, while a conjugation operation costs 2.24 ms more. The net difference between these optima is 4158.36 ms. Equally optimal zero-cost switch arrangements can differ, so diagrams must refer to the particular saved graph rather than an arbitrary later solution.

Refresh count alone also hides a useful trade-off. At attention input 0, the count rises from the baseline's 17 to 18, yet total estimated latency falls 14.6755%. Ordinary-operator costs decrease by 8900.0554 ms while refresh costs increase by 1040.1500 ms. This arithmetic explains the improvement in these saved placements; it is not a causal ablation or proof that every minimum-count solution is worse. Complete MLP input-14 and attention input-0 diagrams are retained with the corresponding graphs.

7. Limitations and remaining work

The evidence supports the encoded rules and fitted costs on two supplied graphs. It does not establish real fully homomorphic encryption (FHE) speed, generalisation to larger or unseen networks, or a runtime ranking against other optimisers. The shared-rule dependencies in our checks remain a limitation. Source, graph and score checks reduce implementation risk but do not independently establish that the course model matches a deployed encrypted system.

Remaining work is to freeze the final method and comparison protocol, run all 30 conditions with formal CPU resource reporting, and provide placement diagrams for at least two input levels per block, checked against saved DAGs. Independent clean-environment reproduction must then resolve necessary discrepancies and feed corrections into the final report. Existing local observations and document checks do not replace these steps.

8. Code and evidence access

The accompanying version README links the optimiser, saved results, evaluation and plotting commands, and exact source revision. It also records the report's evidence and regeneration inputs. The online repository URL and grading revision required for course submission remain to be supplied; this local draft does not establish that online delivery is complete.